

Nama : Farid Aditya Maulana
NIM : 0110224231
Mata Kuliah : Kriptografi

Tugas 9

“Kebocoran Password pada Sistem Akademik Kampus”

Latar Belakang

Sebuah sistem akademik di **sebuah kampus** digunakan untuk:

- Login mahasiswa & admin
- Penyimpanan data nilai
- Informasi keuangan

Namun, sistem tersebut mengalami **insiden kebocoran data**.

Database yang bocor berisi:

- Username
- Password (dalam bentuk hash)

Contoh data bocor:

Username	Password (Hash)
ahmad	5f4dcc3b5aa765d61d8327deb882cf99
ali	e99a18c428cb38d5f260853678922e03
fatimah	25f9e794323b453885f5181f1b624d0b

Permasalahan

Tim IT menemukan bahwa:

- Password di-hash menggunakan **MD5**
- Tidak menggunakan **salt**
- Tidak ada proteksi tambahan

Akibatnya:

- Hacker berhasil melakukan **rainbow table attack**
- Banyak password berhasil ditebak

Pertanyaan

1. Apa fungsi hash dalam keamanan sistem?

Fungsi hash adalah algoritma kriptografi yang mengubah data (seperti password) menjadi string karakter acak dengan panjang tetap yang disebut digest atau hash value. Fungsi utamanya dalam keamanan sistem:

- One-way / tidak bisa dibalik — hash tidak bisa diubah kembali menjadi data asli secara langsung.
- Deterministic — input yang sama selalu menghasilkan hash yang sama.
- Melindungi data di database — sistem tidak menyimpan password asli, melainkan hash-nya. Saat login, password yang diketik akan di-hash lalu dibandingkan dengan hash yang tersimpan.
- Integritas data — memastikan data tidak dimodifikasi (jika data berubah, hash berubah total).

2. Mengapa MD5 tidak lagi aman?

MD5 dianggap tidak aman karena beberapa alasan:

- Collision attack — MD5 terbukti rentan terhadap collision, yaitu dua input berbeda bisa menghasilkan hash yang sama.
- Kecepatan komputasi sangat tinggi — MD5 dirancang untuk kecepatan, bukan keamanan. GPU modern mampu menghitung miliaran hash MD5 per detik, membuat brute force sangat mudah.
- Rainbow table — terdapat database besar berisi pemetaan hash MD5 ke password aslinya, sehingga hash MD5 bisa dibalik hanya dengan pencarian tabel.
- Sudah dinyatakan usang — badan keamanan seperti NIST dan komunitas kriptografi telah merekomendasikan agar MD5 tidak digunakan untuk keperluan keamanan sejak awal 2000-an.

Contoh pada soal: hash **5f4dcc3b5aa765d61d8327deb882cf99** adalah MD5 dari kata **password** — bisa langsung ditemukan di situs seperti crackstation.net.

3. Mengapa penggunaan tanpa salt berbahaya?

Salt adalah nilai acak unik yang ditambahkan ke setiap password sebelum di-hash. Tanpa salt:

- Serangan Rainbow Table menjadi sangat efektif — hacker tinggal mencocokkan hash dalam database dengan tabel yang sudah disiapkan sebelumnya.
- Password yang sama menghasilkan hash yang sama — jika dua user memakai password yang sama (misalnya "123456"), hash mereka identik. Hacker cukup membobol satu, maka semua akun dengan hash sama ikut terbongkar.
- Efisiensi serangan meningkat — tanpa salt, satu password yang berhasil di-crack bisa berlaku untuk banyak akun sekaligus.

Dengan salt, meskipun dua user memiliki password yang sama, hash yang dihasilkan akan berbeda karena salt-nya unik.

4. Bagaimana hacker bisa mengetahui password asli?

Hacker menggunakan beberapa teknik:

- Rainbow Table Attack — menggunakan database yang berisi jutaan pasangan hash ↔ password asli yang sudah dihitung sebelumnya. Tinggal mencari hash yang bocor di tabel tersebut.
- Brute Force Attack — mencoba semua kemungkinan kombinasi karakter, meng-hash setiap kombinasi, lalu membandingkan hasilnya dengan hash target.
- Dictionary Attack — mencoba daftar kata-kata umum, password populer, dan variasinya.
- Online Hash Lookup — memanfaatkan situs seperti crackstation.net atau md5decrypt.net yang menyimpan database hash MD5 umum.

Karena MD5 tanpa salt digunakan pada soal ini, ketiga hash di tabel dapat dengan mudah di-crack:

- **ahmad** → **5f4dcc3b5aa765d61d8327deb882cf99** = password
- **ali** → **e99a18c428cb38d5f260853678922e03** = abc123
- **fatimah** → **25f9e794323b453885f5181f1b624d0b** = 123456789

5. Usulkan algoritma hash yang lebih aman

Berikut algoritma yang direkomendasikan untuk hashing password:

Algoritma	Keterangan
bcrypt	Dirancang khusus untuk password, memiliki faktor cost yang bisa disesuaikan, lambat secara sengaja
Argon2	Pemenang kompetisi Password Hashing Competition (PHC) 2015, sangat direkomendasikan saat ini
scrypt	Membutuhkan memori besar sehingga sulit dijalankan di GPU/ASIC
PBKDF2	Direkomendasikan NIST, banyak digunakan di sistem enterprise

6. Bagaimana cara mengamankan sistem login?

Langkah-langkah pengamanan sistem login yang komprehensif:

Lapisan Hashing:

- Gunakan algoritma aman seperti Argon2id atau bcrypt
- Selalu tambahkan salt unik yang dibuat secara acak untuk setiap password
- Gunakan work factor / cost factor yang cukup tinggi agar lambat di-brute force

Lapisan Autentikasi:

- Terapkan Multi-Factor Authentication (MFA) — misalnya OTP via email/SMS atau aplikasi authenticator

- Batasi percobaan login (rate limiting) — misalnya maksimal 5 kali gagal, lalu akun terkunci sementara
- Gunakan CAPTCHA untuk mencegah serangan otomatis

Lapisan Perlindungan Data:

- Enkripsi database menggunakan AES-256
- Terapkan prinsip least privilege — hanya komponen yang perlu saja yang bisa mengakses tabel password
- Audit dan monitoring log login secara berkala

Lapisan Infrastruktur:

- Gunakan HTTPS/TLS agar data tidak disadap saat transmisi
- Perbarui sistem secara rutin untuk menutup celah keamanan
- Lakukan penetration testing secara berkala